

# Indices

---

## Indices Cluster

El índice se encuentra dentro de los datos, para estos índices se define una estructura, para el ejemplo una árbol binario, pero normalmente son estructuras más complejas, estos árboles tienen un hijo izquierdo y un hijo derecho, dentro del archivo donde se maneja el índice se define un header que posee información de los datos que se están guardando, si el índice se acomoda por números, el header debe decir el row donde inicia el índice, es decir los bytes, y después se tiene un estándar para mostrar cuando hay presencia o ausencia de los hijos derechos o izquierdos. Dependiendo de los datos que se insertan se realizan el acomodo e inserción correcta de los datos (balanceos e inserciones). Se debe hacer una representación de la estructura que se está utilizando dentro del archivo. Cuando se desea realizar un borrado, dentro del header, se tiene un root que apunta a donde arranca la estructura y tenemos, un puntero que apunta a disco, y cuando se borra un elemento se elimina poniéndole un 0, el campo donde se realiza el eliminado, se acomoda todo para tener los datos de una forma más compacta, cuando se formatea el disco, lo que realiza es que al estar metiendo y sacando datos de la BD, se empieza a tener espacios en la BD que quedan libres, y lo que se hace es pegar al final y el SO, creía que los espacios estaban siendo utilizados y que la memoria disminuía cada vez más, al utilizar el método anterior, todo queda más comprimido y el uso del espacio es más eficiente.

## Indices Non Cluster

Este tipo de índice posee la estructura de datos y se tiene de igual forma un header, que funciona exactamente igual al anterior, pero los datos cuando se van ingresando, se colocan de la misma forma, pero no van a estar dentro de los mismos datos. En lenguajes de alto nivel, se tiene el seek, que coloca la cabeza lectora del disco y la coloca donde encuentra el dato, otro concepto es el de definir un índice de expresión, con este se puede generar la estructura de datos, y el índice siempre jala las columnas de la tabla, cuando esto pasa se realiza el concepto de include (pasa cuando el row es bastante grande), Suponiendo que el row pesa 2MG, y lo que se necesita es el lastname y el name, y juntos miden 512 bytes, cuando se realiza esta operación, se tiene que ir a leer el row para recuperar los datos, el row storage es menos eficiente que al utilizar un columnar storage, lo que se hace es que cuando se define el índice, en esta se mete un include de name y lastname, estos datos quedan guardados entonces en el índice y no se debe de buscar los datos en memoria, si no que quedan guardados y si la operación se realiza muchas veces en el sistema, esto mejora mucho los tiempos de respuesta.

## Implicaciones de los índices

Cuando se realiza un insert, si no se poseen índices, cuando se realiza una inserción nueva, lo que se hace es guardar los datos al final de las tablas, (append), cuando no se posee un índice, no se dura nada realizando la inserción, pero las búsquedas van a ser super lentas, ya que realiza una búsqueda lineal. En cambio cuando se tiene un clustered index, las inserciones van a ser rápidas de igual forma, pero no tanto como una inserción con índices, ya que se debe de realizar un insert al índice y a los datos, y el índice al ser una estructura, se deben de seguir todas las reglas que implica una inserción lo que lo hace un poco más lento. Un índice se puede definir sobre solo un set de columnas, solo las consultas que realicen las consultas con estos campos se van a ver beneficiadas. Si se define un non clustered, se pone más lento ya que se debe de realizar el insert en el non cluster, y el insert en el clustered, el select, se vuelve más rápido y solo para ciertas columnas o

consultas, solo las que usan el clustered y el non clustered index, cuando se meten muchos indices, lo empieza a pasar es que baja el rendimiento de la base de datos, entre mas indices, mas disco se necesita, ejemplo, se tiene un indice de 10gb de memoria, y la maquina solo 6, y entonces, cuando este intenta cargar todos los datos en memoria, se va a empezar a realizar transacciones a disco, y se tienen muchos procesos que limitan el rendimiento de las bases de datos, no todo se arregla con indices, muchos indices no es un buen diseño, y cuando se necesitan muchos indices, la memoria debe de crecer con respecto al tamaño de los indices, cuando se tiene un clustered, non clustered, y non clustered con include, pasa lo mismo, se debe de guardar los datos en cada indice y cuando se utiliza un include, el problema involucra a los updates, cuando se realiza un update, dentro del ordenamiento del index, se tiene que rebalancer la estructura de indice, y eso es bastante peligroso, ya que causa que se rebalancee el arbol y puede afectar otras operaciones de la base de datos, y con un include, y se va a realizar un update, el update debe de ir al archivo de datos como al archivo del indice, y se baja el rendimiento de la base de datos incluso un poco mas. En una base de datos SQL, si se debe realizar una carga de datos alto, lo que se realiza es, desactivar los indices, deshabilitar todos, se realizan las operaciones, y despues se corre la operacion de reindex, y mientras esta en esto, quien consulta la base de datos, lo que realiza la base de datos es un scan, esto hace que los inserts y los updates pasen rapido en la base de datos, y se van a liberar los recursos de forma muy rapida, y la persona que realiza los updates e inserts de una forma muy rapida y sin interrupciones, y el usuario que consulta se va a ver afectado en tiempo pero solo durante el tiempo que dure el reindex, al cambiar claves de ordenamiento del indice, se realizan muchas operaciones de rebalanceo y es mejor hacer el reindex ya que realiza la estructura optima de la base de datos, esto es mucho mejor.

## Composed Index

Se puede tener varios enfoques, cuando las columnas que se tienen son strings, se tiene que utilizar la definicion de las columnas como el orden de los campos con las clausulas del where. El ordenamiento del indice se realiza aca. cuando se define el indice con una estructura como el arbol binario, no va a funcionar, entonces se debe de colocar en otra estructura diferente como un b+, donde no se indexa la palabra completa si no que se tiene algo simplificado, pero no es tan sencillo analizar este modelo. Se entra y despues la hoja tiene un puntero a los datos, otra forma es mediante un hash, donde se define la funcion matematica que es la combinacion de los 3 campos, md5 de a b y c y despues se convierte a numerico y se realiza el modulo con algun numero, lo malo es que se debe tener el orden de los datos. Otra forma es una estructura de para cada una de las columnas, lo que pasa es que cuando se realiza el ordenamiento, el indice de la columna apunta a donde puede estar la informacion, indice multidimensional. Hay bases de datos que arman el multidimensional con los datos que se definen, es decir permite el acceso por todos los campos, abc, bac, o sus permutaciones, el problema es que son bastante grandes, y se parecen al indice invertido. Cuando el indice no tienen operaciones compuestas, como un concat, no se deberia usar un indice compuesto sino de expresion, lo que hace que este amarrado a las columnas, y en este caso el indice deberia ser de expresion donde se guarda el resultado de las operaciones compuestas en lugar de los campos. Se debe de analizar muy a profundo los indices a utilizar

## Indice Invertido

Cuando se inserta un documento dentro de la base de datos no SQL se le genera un id, esto documentos pasan a una fase de analisis, donde se quita la informacion que no es relevante para la busqueda no van a ser necesarios. por ejemplo las contracciones, entonces se elimina de los indices, y solo queda la informacion relevante para realizar busquedas y se arma el indice invertido. Se realiza una tokenizacion donde dependen del idioma y el caso de uso, se define la forma es que se desea cortar los textos, se realiza un to lower y se

tokeniza en espacios, dividir las frases en espacios, cuando eso se realiza, se crea un diccionario de terminos, donde se tiene todas las palabras que van a estar en todos los documentos, con un arbol b+ o algun parecido, o un hash, y a estos terminos se le define un identificador, y cada palabra se le asigna un id, y se realiza un posting list, donde se tiene una lista de palabras con los documentos en los que aparece, esto lo realiza todos los motores de busqueda de texto completo, esto la mayoría lo basa en Lucene. Se buscan los terminos presentes en la busqueda y definen a ver si están definidos en la tabla de posting, y entonces se puede verificar los documentos que están asociados a dichas palabras y se agrupan. Después se realiza un ranking con el estudio del lenguaje y dice que tan relevantes son los terminos que se encuentran, por ejemplo queen es más relevante que great, esto al ser más común great que queen, entonces de esta forma se refinan las búsquedas, eso se realiza para hacer esto muchas operaciones matemáticas e IA para rankear en base a esto, esto retorna entonces el documento de mayor interés al principio. Estas bases de datos, NoSQL estas recolectan feedback, que le pide al usuario que le de like o dislike dependiendo de la búsqueda que se realiza, y esto lo utiliza para refinar incluso más los datos que se devuelve, también se utiliza la cantidad de referencia de un documento. Si hay una página que es muy visitada, esto es que es más importante, cuando se dan click en los documentos y mientras más tiempo pase, más relevante va a ser dicho documento y entonces se va a subir su ranking. El documento pasa por tres fases, character filter, donde por ejemplo se eliminan para refinar las búsquedas, igual que los puntos, comas etc, tokenizador por espacios, es decir por palabras, los token filters quitan las palabras que no son relevantes, como or o that, y entonces no se rankean elementos con estas frases, y solo se quedan las palabras que aportan contexto y estas son las que entran en el diccionario de terminos.